CONTRACTED WIND

消费贷项目流程文档

Type your content here, either by copying your text, or by copying your text after the text, select Paste in this content here, either by, select Paste in this box

目录

CONTENTS

1 项目实施过程
Summary of work

2 项目结果汇总
Summary of work

3 项目实施难点
Summary of work

4 面试如何描述
Summary of work

1

项目实施过程

Summary of woSummary of work

项 目 实 施 流 程 概 览

01

数据分析

分析数据中各字段含义以及关系，为后续特征处理整理思路

02

特征处理

不同类型特征处理方案及具体实现方法

03

特征筛选

按相关性排序，筛选一定量特征作为后续模型的基础

04

模型调参

先采用默认参数，根据默认参数准确度结果挑选部分模型进行调参细化，最终得到最优模型

1.1 数据分析

- 字段说明：
  用户基本信息：信用额度、性别、教育、婚姻状况、年龄
  用户行为数据：5-10月的还款状态、账单金额、支付金额
  标签数据：是否违约
- 处理思路：
  ①数据清洗：数据仅包含用户5-10月数据，但是是否违约判定是针对全月份违约情况。因此可能存在用户在5-10月没有出现违约行为，但是是否违约显示“是”，因此需要对这类噪声数据进行修正或删除
  ②文本标签：‘是否违约’
    二值化
  ③文本特征：‘性别’,‘教育’,‘婚姻状况’,还款状态
    - 01编码、数值化
    - 还款状态：数值化/每种状态分类汇总
  ④数值连续特征：‘信用额度’,‘年龄’,账单金额,支付金额
    a.账单金额：存在正值和负值(正值代表欠款，负值代表溢缴款)
      - 处理：每个月正值负值分位点分别统计求平均
      - 合并负值+正值分位点后统一分段
    b.支付金额：5-10月支付金额虽然分布趋势大致相同，但是应使用同一分段标准
      - 分位点求平均后统一分段
    c.‘信用额度’,‘年龄’：正常分段

```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 24 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   信用额度       30000 non-null  int64
 1   性别         30000 non-null  object
 2   教育         30000 non-null  object
 3   婚姻状况       30000 non-null  object
 4   年龄         30000 non-null  int64
 5   10月还款      30000 non-null  object
 6   9月还款       30000 non-null  object
 7   8月还款       30000 non-null  object
 8   7月还款       30000 non-null  object
 9   6月还款       30000 non-null  object
 10  5月还款       30000 non-null  object
 11  10月的账单金额   30000 non-null  int64
 12  9月的账单金额    30000 non-null  int64
 13  8月的账单金额    30000 non-null  int64
 14  7月的账单金额    30000 non-null  int64
 15  6月的账单金额    30000 non-null  int64
 16  5月的账单金额    30000 non-null  int64
 17  10月支付金额    30000 non-null  int64
 18  9月支付金额     30000 non-null  int64
 19  8月支付金额     30000 non-null  int64
 20  7月支付金额     30000 non-null  int64
 21  6月支付金额     30000 non-null  int64
 22  5月支付金额     30000 non-null  int64
 23  是否违约       30000 non-null  object
dtypes: int64(14), object(10)
memory usage: 5.5+ MB
```

1.2 特征处理

①清洗噪声数据

1.自定义函数：找出噪声数据
　　含义：5-10月还款均为延迟，但是否违约='是'的数据标记为噪声数据

2.在原数据data中新增标记噪声数据的列

3.噪声数据占比2.46%，占比较低，故此处作删除处理

4.删除辅助列

```
def get_zaosheng(a,b,c,d,e,f,y):
    if y=='否':
        return 0
    month_lst=[a,b,c,d,e,f]
    for i in [a,b,c,d,e,f]:
        if '延迟' in str(i):
            return 0
    return 1
```

```
data['是否噪声']=data.apply(lambda x:get_zaosheng(x['10月还款'],
                                                  x['9月还款'],
                                                  x['8月还款'],
                                                  x['7月还款'],
                                                  x['6月还款'],
                                                  x['5月还款'],
                                                  x['是否违约']),axis=1)
```

```
#噪声数据占比
len(data[data['是否噪声']==1])/len(data)
```

```
0.0246
```

```
#删除噪声数据代码
zaosheng_data=data[data['是否噪声']==1]
data.drop(index=zaosheng_data.index,inplace=True)
```

```
#删除辅助查找噪声数据列
data.drop(columns='是否噪声',inplace=True)
```

②标签数据二值化

由于标签数据只有‘是’、‘否’两种类型，所以直接使用二值化处理

‘是’——1

‘否’——0

```
data['是否违约']=data['是否违约'].apply(lambda x:1 if x=='是' else 0)
```

③文本特征处理

文本特征：‘性别’,‘教育’,‘婚姻状况’,还款状态

- **‘性别’,‘教育’,‘婚姻状况’**：类别较少，可直接进行01编码、数值化

```
data_info=['性别', '教育', '婚姻状况']
```

```
#01编码
for col in data_info:
    uniq=data[col].unique()
    for i in uniq:
        data[col+'_'+i]=data[col].apply(lambda x:1 if x==i else 0)
```

```
#数值化
for col in data_info:
    uniq=list(data[col].unique())
    data[col]=data[col].apply(lambda x:uniq.index(x))
```

1.2 特征处理

③文本特征处理

文本特征：‘性别’,‘教育’,‘婚姻状况’,还款状态

- **还款状态**：共有5-10月每个月不同状态，且每个月出现的还款状态种类不尽相同
  - a.得到所有状态类型
  - b.按行为恶劣程度人为排序

```
#找到所有状态的可能
huankuan_col=['10月还款', '9月还款', '8月还款', '7月还款','6月还款', '5月还款']
zhuangtai=[]
for col in huankuan_col:
    uniq_lst=list(data[col].unique())
    zhuangtai.extend(uniq_lst)
zhuangtai=list(set(zhuangtai))
```

```
zhuangtai
```

```
['还款延迟不到一个月',
 '还款延迟六个月',
 '还款延迟四个月',
 '提前还款',
 '还款延迟七个月',
 '还款延迟五个月',
 '还款延迟一个月',
 '还款延迟二个月',
 '还款延迟三个月',
 '按时还款',
 '还款延迟八个月']
```

```
#状态行为自定义排序
zhuangtai=['提前还款',
           '按时还款',
           '还款延迟不到一个月',
           '还款延迟一个月',
           '还款延迟二个月',
           '还款延迟三个月',
           '还款延迟四个月',
           '还款延迟五个月',
           '还款延迟六个月',
           '还款延迟七个月',
           '还款延迟八个月']
```

1.2 特征处理

③文本特征处理

文本特征：‘性别’,‘教育’,‘婚姻状况’,还款状态

- **还款状态**：共有5-10月每个月不同状态，且每个月出现的还款状态种类不尽相同
  - c.新增辅助列：合并每个用户5-10月还款状态
  - d.新增11列：每个用户在5-10月期间不同还款状态的分类汇总
  - e.新增列：
    - 延迟还款总次数
    - 按时/提前还款次数
    - 延迟次数占比

```
#得到5-10月还款状态合并文本，便于统计次数
data['还款_strsum']=data['10月还款']+','+data['9月还款']+','+data['8月还款']+','+data['7月还款']+','+data['6月还款']+','+
```

```
#统计1:每个用户5-10月不同还款状态分类汇总
for zt in zhuangtai:
    data[zt+'_次数']=data['还款_strsum'].apply(lambda x:x.count(zt))
```

```
#统计2:延迟还款次数、没有延迟次数、延迟还款次数占比
data['延迟还款次数']=data[['还款延迟不到一个月_次数',
       '还款延迟二个月_次数', '还款延迟三个月_次数', '还款延迟四个月_次数', '还款延迟五个月_次数', '还款延迟六个月_次数',
       '还款延迟七个月_次数', '还款延迟八个月_次数']].sum(axis=1)
data['按时/提前还款次数']=6-data['延迟还款次数']
data['延迟次数占比']=data['延迟还款次数']/6
```

1.2 特征处理

③文本特征处理

文本特征：‘性别’,‘教育’,‘婚姻状况’,还款状态

- **还款状态**：共有5-10月每个月不同状态，且每个月出现的还款状态种类不尽相同
  - f.延迟还款状态分段统计(不同延迟时间排列组合)
    - 分类1:不到1个月-1个月，延迟2-3月，2-4月，2-5月，2-6月，2-7月，2-8月
    - 分类2:不到1个月-2个月，延迟3-4月，3-5月，3-6月，3-8月
    - 分类3:不到1个月-3个月，延迟4-5月，4-6月，4-7月，4-8月
    - 分类4:不到1个月-4个月，延迟5-6月，5-7月，5-8月
    - 分类5:不到1个月-5个月，延迟6-7月，6-8月
    - 分类6:不到1个月-6个月，延迟7-8月
    - 分类7:不到1个月-7个月

```
#分类1
data['延迟_1']=data[['还款延迟不到一个月_次数','还款延迟一个月_次数']].sum(axis=1)
data['延迟_2-3']=data[['还款延迟二个月_次数','还款延迟三个月_次数']].sum(axis=1)
data['延迟_2-4']=data[['还款延迟二个月_次数','还款延迟三个月_次数','还款延迟四个月_次数']].sum(axis=1)
data['延迟_2-5']=data[['还款延迟二个月_次数','还款延迟三个月_次数','还款延迟四个月_次数','还款延迟五个月_次数']].sum(axis=1)
data['延迟_2-6']=data[['还款延迟二个月_次数','还款延迟三个月_次数','还款延迟四个月_次数','还款延迟五个月_次数','还款延迟六个月_次
data['延迟_2-7']=data[['还款延迟二个月_次数','还款延迟三个月_次数','还款延迟四个月_次数','还款延迟五个月_次数','还款延迟六个月_次
data['延迟_2-8']=data[['还款延迟二个月_次数','还款延迟三个月_次数','还款延迟四个月_次数','还款延迟五个月_次数','还款延迟六个月_次
```

```
#分类2
data['延迟_2']=data[['还款延迟不到一个月_次数','还款延迟一个月_次数','还款延迟二个月_次数']].sum(axis=1)
data['延迟_3-4']=data[['还款延迟三个月_次数','还款延迟四个月_次数']].sum(axis=1)
data['延迟_3-5']=data[['还款延迟三个月_次数','还款延迟四个月_次数','还款延迟五个月_次数']].sum(axis=1)
data['延迟_3-6']=data[['还款延迟三个月_次数','还款延迟四个月_次数','还款延迟五个月_次数','还款延迟六个月_次数']].sum(axis=1)
data['延迟_3-7']=data[['还款延迟三个月_次数','还款延迟四个月_次数','还款延迟五个月_次数','还款延迟六个月_次数','还款延迟七个月_次
data['延迟_3-8']=data[['还款延迟三个月_次数','还款延迟四个月_次数','还款延迟五个月_次数','还款延迟六个月_次数','还款延迟七个月_次
```

```
#分类3
data['延迟_3']=data[['还款延迟不到一个月_次数','还款延迟一个月_次数','还款延迟二个月_次数','还款延迟三个月_次数']].sum(axis=1)
data['延迟_4-5']=data[['还款延迟四个月_次数','还款延迟五个月_次数']].sum(axis=1)
data['延迟_4-6']=data[['还款延迟四个月_次数','还款延迟五个月_次数','还款延迟六个月_次数']].sum(axis=1)
data['延迟_4-7']=data[['还款延迟四个月_次数','还款延迟五个月_次数','还款延迟六个月_次数','还款延迟七个月_次数']].sum(axis=1)
data['延迟_4-8']=data[['还款延迟四个月_次数','还款延迟五个月_次数','还款延迟六个月_次数','还款延迟七个月_次数','还款延迟八个月_次数
```

```
#分类4
data['延迟_4']=data[['还款延迟不到一个月_次数','还款延迟一个月_次数','还款延迟二个月_次数','还款延迟三个月_次数','还款延迟四个月_次
data['延迟_5-6']=data[['还款延迟五个月_次数','还款延迟六个月_次数']].sum(axis=1)
data['延迟_5-7']=data[['还款延迟五个月_次数','还款延迟六个月_次数','还款延迟七个月_次数']].sum(axis=1)
data['延迟_5-8']=data[['还款延迟五个月_次数','还款延迟六个月_次数','还款延迟七个月_次数','还款延迟八个月_次数']].sum(axis=1)
```

```
#分类5
data['延迟_5']=data[['还款延迟不到一个月_次数','还款延迟一个月_次数','还款延迟二个月_次数','还款延迟三个月_次数','还款延迟四个月_次
data['延迟_6-7']=data[['还款延迟六个月_次数','还款延迟七个月_次数']].sum(axis=1)
data['延迟_6-8']=data[['还款延迟六个月_次数','还款延迟七个月_次数','还款延迟八个月_次数']].sum(axis=1)
```

③文本特征处理

文本特征：‘性别’,‘教育’,‘婚姻状况’,还款状态

- **还款状态**：共有5-10月每个月不同状态，且每个月出现的还款状态种类不尽相同

g.原还款状态列数值化

h.删除辅助列

```
huankuan_col
```

```
['10月还款', '9月还款', '8月还款', '7月还款', '6月还款', '5月还款']
```

```
zhuangtai
```

```
['提前还款',
 '按时还款',
 '还款延迟不到一个月',
 '还款延迟一个月',
 '还款延迟二个月',
 '还款延迟三个月',
 '还款延迟四个月',
 '还款延迟五个月',
 '还款延迟六个月',
 '还款延迟七个月',
 '还款延迟八个月']
```

```
for col in huankuan_col:
    data[col]=data[col].apply(lambda x:zhuangtai.index(x))
```

```
#删除辅助列：还款_strsum
data.drop(columns='还款_strsum',inplace=True)
```

④数值特征处理

数值连续特征：‘信用额度’,‘年龄’,账单金额,支付金额

账单金额：存在正值和负值(正值代表欠款，负值代表溢缴款)

a.查看数据分布情况：

<0的数据占比不足10%，分段时应分类讨论

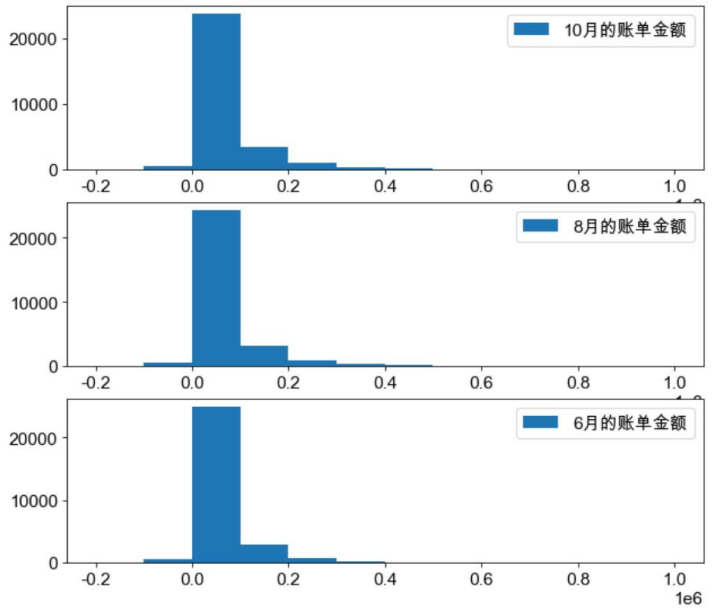
>0的数据呈现明显拖尾，前多后少

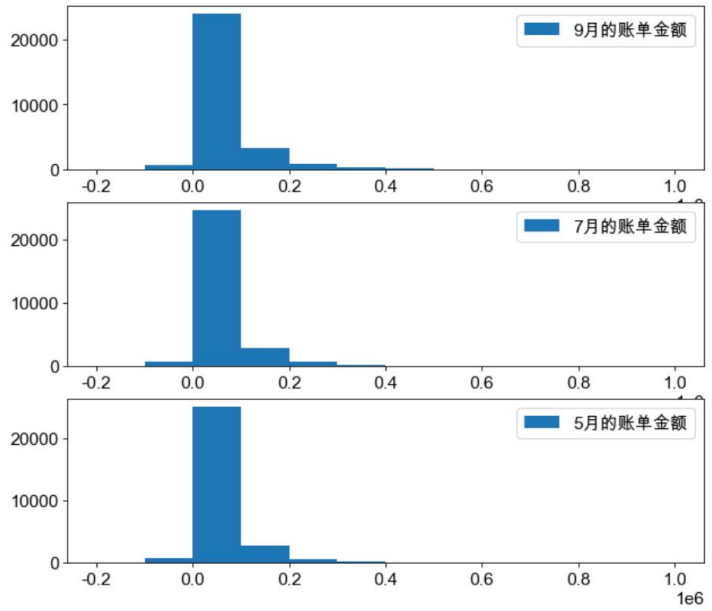
各月份账单金额分布趋势大致相同

```
zhangdan_col=['10月的账单金额', '9月的账单金额', '8月的账单金额', '7月的账单金额', '6月的账单金额', '5月的账单金额']
```

```
#各月账单分布情况(describe显示负数数据不足10%)
data[zhangdan_col].describe(percentiles=np.linspace(0.1,0.9,9))
```

	10月的账单金额	9月的账单金额	8月的账单金额	7月的账单金额	6月的账单金额	5月的账单金额
count	29262.000000	29262.000000	2.926200e+04	29262.000000	29262.000000	29262.000000
mean	52395.132356	50327.713758	4.811053e+04	44270.324653	41253.394197	39770.382100
std	74100.992370	71629.102702	6.981803e+04	64768.237386	61234.242939	59950.479628
min	-165580.000000	-69777.000000	-1.572640e+05	-170000.000000	-81334.000000	-209051.000000
10%	316.000000	76.100000	0.000000e+00	0.000000	0.000000	0.000000
20%	2280.000000	1795.000000	1.479200e+03	1217.400000	918.200000	604.200000
30%	6892.000000	6378.000000	6.152000e+03	5543.300000	4450.600000	3274.000000
40%	14702.800000	14192.800000	1.352340e+04	12440.800000	10575.400000	9538.600000
50%	24018.000000	22906.000000	2.108500e+04	19592.500000	18726.000000	17899.000000
60%	39002.400000	36628.600000	3.301860e+04	29263.400000	27668.200000	26584.800000
70%	54308.200000	51583.400000	4.991310e+04	46757.800000	42605.000000	40522.400000
80%	85553.200000	81990.600000	7.829700e+04	72276.600000	67538.800000	65191.200000
90%	144243.500000	138541.000000	1.335457e+05	124489.500000	117788.600000	114731.900000
max	964511.000000	983931.000000	1.664089e+06	891586.000000	927171.000000	961664.000000





④数值特征处理

数值连续特征：‘信用额度’,‘年龄’,账单金额,支付金额

账单金额：存在正值和负值(正值代表欠款，负值代表溢缴款)

b.分别筛选账单金额<0的四分位点和>0的十分位点，多月分位点求平均

```
#每个月账单金额为负的四分位点及其均值
data_zd0jian=pd.DataFrame(index=['count', 'mean', 'std', 'min', '25%', '50%', '75%', 'max'])
for col in zhangdan_col:
    data_zd0jian[col]=data[col][data[col]<0].describe()
data_zd0jian['5-10月账单金额平均']=data_zd0jian.mean(axis=1)
data_zd0jian
```

```
#每个月账单金额为正的十位点及其均值
data_zd0jia=pd.DataFrame(index=['count', 'mean', 'std', 'min', '10%', '20%', '30%', '40%', '50%', '60%',
                                '70%', '80%', '90%', 'max'])
for col in zhangdan_col:
    data_zd0jia[col]=data[col][data[col]>0].describe(percentiles=np.linspace(0.1,0.9,9))
data_zd0jia['5-10月账单金额平均']=data_zd0jia.mean(axis=1)
data_zd0jia
```

④数值特征处理

数值连续特征：‘信用额度’,‘年龄’,账单金额,支付金额

账单金额：存在正值和负值(正值代表欠款，负值代表溢缴款)

c.合并分位点：合并5-10月<0的平均四分位点与>0的平均十分位点，得到13个终极分位点

```
#合并分位点,共13个分位点，可以划分成14分段
fenweidian_fu=data_zd0jian['5-10月账单金额平均'][[ '25%', '50%', '75%' ,'max']]
fenweidian_zheng=data_zd0jia['5-10月账单金额平均'][[ '10%', '20%', '30%', '40%', '50%', '60%',
       '70%', '80%', '90%']]
zhangdan_fenweidian=list(pd.concat((fenweidian_fu,fenweidian_zheng)).values)
zhangdan_fenweidian
```

```
[-837.7916666666666,
 -156.83333333333334,
 -16.291666666666668,
 -1.0,
 1397.0833333333333,
 5014.966666666667,
 10698.450000000003,
 18207.533333333336,
 26393.666666666668,
 38764.2,
 54841.34999999997,
 84180.56666666671,
 138316.9]
```

1.2 特征处理

④数值特征处理

数值连续特征：‘信用额度’,‘年龄’,账单金额,支付金额

账单金额：存在正值和负值(正值代表欠款，负值代表溢缴款)

d.分段：自定义分段函数，再对原5-10月账单金额进行分段

```
#自定义14分段函数
def get_cut14(x,cut14_1,cut14_2,cut14_3,cut14_4,cut14_5,cut14_6,cut14_7,
              cut14_8,cut14_9,cut14_10,cut14_11,cut14_12,cut14_13):
    if x<cut14_1:
        return 1
    elif x<cut14_2:
        return 2
    elif x<cut14_3:
        return 3
    elif x<cut14_4:
        return 4
    elif x<cut14_5:
        return 5
    elif x<cut14_6:
        return 6
    elif x<cut14_7:
        return 7
    elif x<cut14_8:
        return 8
    elif x<cut14_9:
        return 9
    elif x<cut14_10:
        return 10
    elif x<cut14_11:
        return 11
    elif x<cut14_12:
        return 12
    elif x<cut14_13:
        return 13
    else:
        return 14
```

```
cut14_1,cut14_2,cut14_3,cut14_4,cut14_5,cut14_6,cut14_7,cut14_8,cut14_9,cut14_10,cut14_11,cut14_12,cut14_13=zhangdan
```

```
#5-10月账单金额14分段
for col in zhangdan_col:
    data[col]=data[col].apply(lambda x:get_cut14(x,cut14_1,cut14_2,cut14_3,cut14_4,cut14_5,cut14_6,cut14_7,
                                                 cut14_8,cut14_9,cut14_10,cut14_11,cut14_12,cut14_13))
```

- 支付金额字段+信息额度+年龄分段化

1.2 特征处理

④数值特征处理

数值连续特征：‘信用额度’,‘年龄’,账单金额,支付金额

支付金额+‘信用额度’,‘年龄’： 最终分段方案一致故合并处理

a.得到5-10月支付金额平均四分位点和十分位点

```
zhifu_col=['10月支付金额', '9月支付金额', '8月支付金额', '7月支付金额', '6月支付金额', '5月支付金额']
```

```
zhifu_des4=data[zhifu_col].describe()
zhifu_des4['5-10月支付均值']=zhifu_des4.mean(axis=1)
zhifu_des4
```

```
zhifu_des10=data_zhifu.describe(percentiles=np.linspace(0.1,0.9,9))
zhifu_des10['5-10月支付均值']=zhifu_des10.mean(axis=1)
zhifu_des10
```

④数值特征处理

数值连续特征：‘信用额度’,‘年龄’,账单金额,支付金额

支付金额+‘信用额度’,‘年龄’： 最终分段方案一致故合并处理

b.得到‘信用额度’、‘年龄’四分位点和十分位点

c.合并：支付金额字段平均分位点，‘信用额度’、‘年龄分位点

```
xinxi_des4=data[['信用额度', '年龄']].describe()
xinxi_des4
```

```
xinxi_des10=data[['信用额度', '年龄']].describe(np.linspace(0.1,0.9,9))
xinxi_des10
```

```
cut_des4=pd.concat((zhifu_des4['5-10月支付均值'],xinxi_des4),axis=1)
cut_des4
```

```
cut_des10=pd.concat((zhifu_des10['5-10月支付均值'],xinxi_des10),axis=1)
cut_des10
```

1.2 特征处理

④数值特征处理

数值连续特征：‘信用额度’,‘年龄’,账单金额,支付金额

支付金额+‘信用额度’,‘年龄’： 最终分段方案一致故合并处理

d.自定义四分段+十分段函数

e.分段

```
def get_cut4(x,cut4_1,cut4_2,cut4_3):
    if x<cut4_1:
        return 1
    elif x<cut4_2:
        return 2
    elif x<cut4_3:
        return 3
    else:
        return 4
```

```
def get_cut10(x,cut10_1,cut10_2,cut10_3,cut10_4,cut10_5,cut10_6,cut10_7,cut10_8,cut10_9):
    if x<cut10_1:
        return 1
    elif x<cut10_2:
        return 2
    elif x<cut10_3:
        return 2
    elif x<cut10_4:
        return 3
    elif x<cut10_5:
        return 5
    elif x<cut10_6:
        return 6
    elif x<cut10_7:
        return 7
    elif x<cut10_8:
        return 8
    elif x<cut10_9:
        return 9
    else:
        return 10
```

```
cut4_10_col=['信用额度', '年龄', '10月支付金额', '9月支付金额', '8月支付金额', '7月支付金额', '6月支付金额', '5月支付金额']
```

```
for col in cut4_10_col:
    #四分段
    if  '支付' in col:
        cut4_1,cut4_2,cut4_3=cut_des4['5-10月支付均值'][['25%', '50%', '75%']]
    else:
        cut4_1,cut4_2,cut4_3=cut_des4[col][['25%', '50%', '75%']]
    data[col+'_4']=data[col].apply(lambda x:get_cut4(x,cut4_1,cut4_2,cut4_3))

    #十分段
    if  '支付' in col:
        cut10_1,cut10_2,cut10_3,cut10_4,cut10_5,cut10_6,cut10_7,cut10_8,cut10_9=cut_des10['5-10月支付均值'][['10%', '
                                                                                                                '60%', '70%'
    else:
        cut10_1,cut10_2,cut10_3,cut10_4,cut10_5,cut10_6,cut10_7,cut10_8,cut10_9=cut_des10[col][['10%', '20%', '30%',
                                                                                                '70%', '80%', '90%',]
    data[col+'_10']=data[col].apply(lambda x:get_cut10(x,cut10_1,cut10_2,cut10_3,cut10_4,cut10_5,cut10_6,cut10_7,cut
```

1.3 特征筛选

1.使用corr()函数得到所有特征间相关系数：data_corr=data.corr()

2.筛选出y标签(AM)与各特征之间的相关系数绝对值和排序：

y_corr=data_corr['AM'].abs().sort_values(ascending=False)

3.筛选出相关系数>0.1的特征：

list(y_corr[y_corr>0.1].index

4.手动剔除相同特征的不同分段类型，保留相关性最高的字段

最终：features_all=['延迟_2-7','延迟_2-8','延迟_2-4','延迟_2-6', '延迟_2-5','延迟_2-3','还款延迟二个月_次数','10月还款','9月还款','8月还款','7月还款','6月还款','5月还款','信用额度_4','还款延迟三个月_次数','延迟_3-7','延迟_3-4','延迟_3-8','延迟_7','延迟_3-5', '延迟_3-6','延迟_6','延迟_5','延迟_4','延迟_3','延迟次数占比','延迟还款次数','按时/提前还款次数','延迟_2','按时还款_次数','10月支付金额_4','还款延迟一个月_次数','9月支付金额_4','延迟_4-7','延迟_4-8','8月支付金额_4','提前还款_次数', '5月支付金额_10']

len(features_all)=38

5.划分训练集/测试集：x1,x2,y1,y2=train_test_split(data[features_all],data['是否违约'], train_size=0.8,stratify=data['是否违约'])

```
list(y_corr[y_corr>0.1].index)
```

```
['是否违约',
 '延迟_2-7',
 '延迟_2-8',
 '延迟_2-4',
 '延迟_2-6',
 '延迟_2-5',
 '延迟_2-3',
 '还款延迟二个月_次数',
 '10月还款',
 '9月还款',
 '8月还款',
 '7月还款',
 '6月还款',
 '5月还款',
 '信用额度_4',
 '还款延迟三个月_次数',
 '信用额度_10',
 '延迟_3-7',
 '延迟_3-4',
 '延迟_3-8',
 '延迟_7',
 '延迟_3-5',
 '延迟_3-6',
 '延迟_6',
 '信用额度',
 '延迟_5',
 '延迟_4',
 '延迟_3',
 '延迟次数占比',
 '延迟还款次数',
 '按时/提前还款次数',
 '延迟_2',
 '按时还款_次数',
 '10月支付金额_4',
 '还款延迟一个月_次数',
 '10月支付金额_10',
 '9月支付金额_4',
 '9月支付金额_10',
 '延迟_4-7',
 '延迟_4-8',
 '8月支付金额_4',
 '8月支付金额_10',
 '提前还款_次数',
 '5月支付金额_10']
```

1.4 模型调参

①默认参数运行

模型准确度排序

逻辑回归>Adaboost>0.84,因此后续选择对这两个模型进行调参

此外再增加随机森林模型调参

(1) 逻辑回归

```
lr_model=LR().fit(x1,y1)
lr_model.score(x2,y2)
```

```
0.8419613873227405
```

```
lr_model=LR()
cross_s=cross_val_score(lr_model,data[features_all],data['是否违约'],cv=5)
cross_s,cross_s.mean()
```

```
(array([0.82555954, 0.83051427, 0.84039645, 0.84501025, 0.84244703]),
 0.836785506705116)
```

(2) KNN

```
knn_model=KNeighborsClassifier().fit(x1,y1)
knn_model.score(x2,y2)
```

```
0.8274389202118572
```

(3) 朴素贝叶斯

```
gs_model=GaussianNB().fit(x1,y1)
gs_model.score(x2,y2)
```

```
0.8305142661882795
```

```
bnl_model=BernoulliNB().fit(x1,y1)
bnl_model.score(x2,y2)
```

```
0.7903639159405433
```

```
mul_model=MultinomialNB().fit(x1,y1)
mul_model.score(x2,y2)
```

```
0.8147958311976764
```

(4) 决策树

```
dt_model=DecisionTreeClassifier().fit(x1,y1)
dt_model.score(x2,y2)
```

```
0.8069366137023749
```

(5) 随机森林

```
rf_model=RandomForestClassifier().fit(x1,y1)
rf_model.score(x2,y2)
```

```
0.8286348880915769
```

(6) Adaboost

```
ada_model=AdaBoostClassifier().fit(x1,y1)
ada_model.score(x2,y2)
```

```
0.8470869639501111
```

```
ada_model=AdaBoostClassifier()
cross_s=cross_val_score(ada_model,data[features_all],data['是否违约'],cv=5)
cross_s,cross_s.mean()
```

```
(array([0.82555954, 0.82846404, 0.84466849, 0.85269993, 0.84586466]),
 0.8394513320718506)
```

②逻辑回归调参

a.调参

```
result1=[]
for p in {'l1', 'l2', 'elasticnet', 'none'}:
    for i in [True,False]:
        for s in {'newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga'}:
            for mul in {'auto', 'ovr', 'multinomial'}:
                try:
                    lr_model=LR(penalty=p,
                                fit_intercept=i,
                                solver=s,
                                multi_class=mul,
                                max_iter=10000).fit(x1,y1)
                    d={'penalty':p,
                       'fit_intercept':i,
                       'solver':s,
                       'multi_class':mul,
                      'score':lr_model.score(x2,y2)}
                    result1.append(d)
                except ValueError:
                    continue
```

②逻辑回归调参

b.筛选最优模型

```
result1_df=pd.DataFrame(result1).sort_values(by='score',ascending=False)
result1_df
```

	penalty	fit_intercept	solver	multi_class	score
0	l2	True	liblinear	auto	0.842132
30	none	True	newton-cg	ovr	0.842132
34	none	True	sag	multinomial	0.842132
35	none	True	sag	auto	0.842132
36	none	True	sag	ovr	0.842132
...	...	...	...	...	...
28	none	True	newton-cg	multinomial	0.842132
61	l1	False	saga	ovr	0.842132
32	none	True	lbfgs	auto	0.841961
5	l2	True	lbfgs	multinomial	0.841961
33	none	True	lbfgs	ovr	0.841961

62 rows × 5 columns

```
result1_df[(result1_df['score']>0.842)
          &(result1_df['penalty']=='l2')
          &(result1_df['fit_intercept']==True)
          &(result1_df['multi_class']=='ovr')
          ]
```

	penalty	fit_intercept	solver	multi_class	score
1	l2	True	liblinear	ovr	0.842132
4	l2	True	newton-cg	ovr	0.842132
7	l2	True	lbfgs	ovr	0.842132
10	l2	True	sag	ovr	0.842132
13	l2	True	saga	ovr	0.842132

②逻辑回归调参

c.最优模型+交叉验证

与最初默认参数模型相比准确度提升不大

逻辑回归最优模型准确度:

```
lr_best_model=LR(penalty='l2',
                 fit_intercept=True,
                 solver='liblinear',
                 multi_class='ovr',
                 max_iter=100000).fit(x1,y1)
lr_best_model.score(x2,y2)
```

```
0.8421322398769862
```

```
lr_best_model=LR(penalty='l2',
                 fit_intercept=True,
                 solver='liblinear',
                 multi_class='ovr',
                 max_iter=100000)
cross_s=cross_val_score(lr_best_model,data[features_all],data['是否违约'],cv=5)
cross_s,cross_s.mean()
```

```
(array([0.82555954, 0.83000171, 0.84022556, 0.84501025, 0.84261791]),
 0.8366829951725686)
```

1.4 模型调参

③Adaboost调参

Adaboost基分类器：逻辑回归
a.调参：最优模型结果准确度0.842645，略高于逻辑回归模型

```
result2=[]
for p in {'l1', 'l2', 'elasticnet', 'none'}:
    for i in [True,False]:
        for s in {'newton-cg', 'lbfgs', 'liblinear', 'sag', 'saga'}:
            for mul in {'auto', 'ovr', 'multinomial'}:
                for al in {'SAMME', 'SAMME.R'}:
                try:
    lr_model=LR(penalty=p,
                fit_intercept=i,
                solver=s,
                multi_class=mul,
                max_iter=10000)
    ada_model=AdaBoostClassifier(base_estimator=lr_model,
                                 n_estimators=50,
                                 algorithm=al).fit(x1,y1)
    d={'penalty':p,
       'fit_intercept':i,
       'solver':s,
       'multi_class':mul,
       'algorithm':al,
      'score':ada_model.score(x2,y2)}
    result2.append(d)
except ValueError:
    continue
```

```
result2_df=pd.DataFrame(result2).sort_values(by='score',ascending=False)
result2_df
```

	penalty	fit_intercept	solver	multi_class	algorithm	score
15	l2	True	lbfgs	ovr	SAMME.R	0.842645
13	l2	True	lbfgs	auto	SAMME.R	0.842645
53	l2	False	saga	auto	SAMME.R	0.842474
55	l2	False	saga	ovr	SAMME.R	0.842474
7	l2	True	newton-cg	auto	SAMME.R	0.842474
...	...	...	...	...	...	...
108	l1	True	saga	multinomial	SAMME	0.798394
106	l1	True	liblinear	ovr	SAMME	0.798394
105	l1	True	liblinear	auto	SAMME.R	0.798394
104	l1	True	liblinear	auto	SAMME	0.798394
123	l1	False	saga	ovr	SAMME.R	0.798394

124 rows × 6 columns

③Adaboost调参

Adaboost基分类器：逻辑回归
b.最优模型+交叉验证

```
#Adaboost最优模型
lr_best_model=LR(penalty='l2',
            fit_intercept=True,
            solver='lbfgs',
            multi_class='ovr',
            max_iter=100000)
ada_best_model=AdaBoostClassifier(base_estimator=lr_best_model,
                                  n_estimators=50,
                                  algorithm='SAMME.R').fit(x1,y1)
ada_best_model.score(x2,y2)
```

```
0.8426447975397232
```

```
#Adaboost最优模型交叉验证
lr_best_model=LR(penalty='l2',
            fit_intercept=True,
            solver='lbfgs',
            multi_class='ovr',
            max_iter=100000)
ada_best_model=AdaBoostClassifier(base_estimator=lr_best_model,
                                  n_estimators=50,
                                  algorithm='SAMME.R')
cross_s=cross_val_score(ada_best_model,data[features_all],data['是否违约'],cv=5)
cross_s,cross_s.mean()
```

```
(array([0.82624295, 0.82966   , 0.83749146, 0.8436432 , 0.84090909]),
 0.8355893402954291)
```

1.4 模型调参

④随机森林调参

a.调参一：准确度高于0.8457的模型参数分布如下

```
#调参1
result3=[]
for cri in {"gini", "entropy"}:
    for max_d in range(3,21,3):
        for min_s in range(10,100,10):
            for min_l in range(5,50,5):
                if min_s>=2*min_l:
                    rf_model=RandomForestClassifier(n_estimators=30,
                                                    oob_score=True,
                                                    criterion=cri,
                                                    max_depth=max_d,
                                                    min_samples_split=min_s,
                                                    min_samples_leaf=min_l).fit(x1,y1)
                    d={'n_estimators':50,
                       'oob_score':True,
                       'criterion':cri,
                       'max_depth':max_d,
                       'min_samples_split':min_s,
                       'min_samples_leaf':min_l,
                       'score':rf_model.score(x2,y2)}
                    result3.append(d)
```

```
result3_df[result3_df['score']>0.8457].describe()
```

	n_estimators	max_depth	min_samples_split	min_samples_leaf	score
count	9.0	9.000000	9.000000	9.000000	9.000000
mean	50.0	9.333333	78.888889	33.333333	0.845777
std	0.0	4.092676	16.914819	9.013878	0.000121
min	50.0	6.000000	40.000000	15.000000	0.845720
25%	50.0	6.000000	70.000000	30.000000	0.845720
50%	50.0	9.000000	90.000000	35.000000	0.845720
75%	50.0	12.000000	90.000000	40.000000	0.845720
max	50.0	18.000000	90.000000	45.000000	0.846062

```
result3_df=pd.DataFrame(result3).sort_values(by='score',ascending=False)
result3_df
```

	n_estimators	oob_score	criterion	max_depth	min_samples_split	min_samples_leaf	score
403	50	True	entropy	9	90	40	0.846062
87	50	True	gini	6	90	35	0.845891
439	50	True	entropy	12	80	35	0.845720
71	50	True	gini	6	70	30	0.845720
70	50	True	gini	6	70	25	0.845720
...	...	...	...	...	...	...	...
278	50	True	entropy	3	40	15	0.832394
298	50	True	entropy	3	80	5	0.832223
307	50	True	entropy	3	90	10	0.832223
289	50	True	entropy	3	60	25	0.831710
270	50	True	entropy	3	10	5	0.831539

540 rows × 7 columns

1.4 模型调参

④随机森林调参

b.调参二：根据第一次调参结果进行第二次调参，小范围精查找

c.最优模型+交叉验证

```
#随机森林最优模型
rf_best_model=RandomForestClassifier(n_estimators=100,
                                     oob_score=True,
                                     criterion='entropy',
                                     max_depth=8,
                                     min_samples_split=91,
                                     min_samples_leaf=37).fit(x1,y1)
rf_best_model.score(x2,y2)
```

```
0.8455492909618999
```

```
#随机森林最优模型交叉检验
rf_best_model=RandomForestClassifier(n_estimators=100,
                                     oob_score=True,
                                     criterion='entropy',
                                     max_depth=8,
                                     min_samples_split=91,
                                     min_samples_leaf=37)
cross_s=cross_val_score(rf_best_model,data[features_all],data['是否违约'],cv=5)
cross_s,cross_s.mean()
```

```
(array([0.82897659, 0.83205194, 0.84398496, 0.85269993, 0.84569378]),
 0.8406814412668361)
```

```
#调参2
result3_2=[]
for cri in {"gini", "entropy"}:
    for max_d in range(5,19):
        for min_s in range(35,95):
            for min_l in range(15,46):
                if min_s>=2*min_l:
                    rf_model=RandomForestClassifier(n_estimators=30,
                                                    oob_score=True,
                                                    criterion=cri,
                                                    max_depth=max_d,
                                                    min_samples_split=min_s,
                                                    min_samples_leaf=min_l).fit(x1,y1)
                    d={'n_estimators':50,
                       'oob_score':True,
                       'criterion':cri,
                       'max_depth':max_d,
                       'min_samples_split':min_s,
                       'min_samples_leaf':min_l,
                       'score':rf_model.score(x2,y2)}
                    result3_2.append(d)
```

```
result3_2_df=pd.DataFrame(result3_2).sort_values(by='score',ascending=False)
result3_2_df
```

	n_estimators	oob_score	criterion	max_depth	min_samples_split	min_samples_leaf	score
19266	50	True	entropy	8	91	37	0.847087
4025	50	True	gini	8	85	37	0.846916
17037	50	True	entropy	6	89	21	0.846916
15560	50	True	entropy	5	73	36	0.846916
151	50	True	gini	5	54	19	0.846916
...	...	...	...	...	...	...	...
15813	50	True	entropy	5	84	17	0.836665
956	50	True	gini	5	91	19	0.836665
15890	50	True	entropy	5	86	38	0.836494
16096	50	True	entropy	5	93	33	0.836494
15192	50	True	entropy	5	52	20	0.836152

30128 rows × 7 columns

2

项目结果汇总

Summary of woSummary of work

2. 项目结果汇总

逻辑回归:

测试准确度：0.8421322398769862
cv准确度：0.8366829951725686
差值：0.0054492447044417544

Adaboost:

测试准确度：0.8426447975397232
cv准确度：0.8355893402954291
差值：0.0070554572442941 28

随机森林:

测试准确度：0.8455492909618999
cv准确度：0.8406814412668361
差值：0.0048678496950638195

cv准确度排名：随机森林>逻辑回归>Adaboost
泛化性排名：随机森林>逻辑回归>Adaboost

因此最终选择随机森林模型来实现用户违约风险预测，辅助企业降低坏账风险

3

项目实施难点

Summary of woSummary of work

3. 项目实施难点

- 数据清洗过程
  - 刚开始疏忽了数据与业务逻辑，看着数据没什么空缺值就往下做，结果因数据的不完整性带来的“脏数据”直接影响最终结果，深入了解业务逻辑后才明白数据修正的重要性
- **特征工程过程**
  - 处理还款延迟部分时，最初直接套用模版进行01编码，以每个月该用户有无特定状态分列，结果导致新增了几十列，但是数据相关性并无明显提升，对最终模型迭代造成了很大的负担。最后还是从业务出发，了解清楚字段背后的行为逻辑，还款状态粗分无非延迟和不延迟，延迟的话又有不同时长，按照现实生活逻辑判断，经常延迟很久的人违约风险肯定更高，因此在后来的特征工程中以行为状态为基础进行分类汇总后模型准确度明显提高了一点
- **特征筛选过程**
  - 把握不好筛选的度，怕筛多了带来运行负担，筛少了会欠拟合
- **模型选择**
  - 算力和时间问题只选择了部分模型进行深度调参，可能错过更优模型

4

面试如何描述

Summary of woSummary of work

4. 面试如何描述

该项目是封控场景的违约预测模型，核心背景是在信贷业务中，用户违约会带来坏账风险，单靠人工审核工作量大且难以统一标准，因此数据模型可辅助企业识别风险用户，我的目标是基于用户历史数据，构建一个泛化性好、可解释性强的违约预测模型。

经过数据清洗、类别特征编码、数据集划分后，搭建逻辑回归、随机森林、AdaBoost三类二分类模型。采用 5 折交叉验证开展参数寻优，并量化监控模型过拟合情况。综合对比预测精度与泛化稳定性，最终选取随机森林作为预测模型，实现客户违约风险预判，为金融信贷风控提供决策支撑。

CONTRACTED WIND

谢谢您的观看

Type your content here, either by copying your text, or by copying your text after the text, select Paste in this content here, either by, select Paste in this box